

AD 502 – Machine Learning Lab

Pre-Reading Material – Lecture 2

Introduction to Numpy Library

Estimated Reading Time: 25–30 minutes

Reference Reading: Daniel Bourke, ‘A Quick Introduction to Data Analysis with Python and pandas’.

Why This Topic Matters

Machine learning involves working with large amounts of numerical data, and efficient numerical computation is essential for building ML models. **NumPy** provides powerful tools for creating, manipulating, analyzing, and performing mathematical operations on multidimensional arrays.

Learning Outcomes

After completing this pre-reading, you should be able to:

- Explain the basic purpose of **NumPy** and its ndarray structure.
- Understand important array attributes such as **shape, dimension, size, and data type**.
- Create and access NumPy arrays.
- Perform basic arithmetic and aggregation operations on arrays.
- Understand reshaping, transposing, sorting, and dot products.
- Explain how NumPy arrays can be used to represent image data.

Activate Your Prior Knowledge

1. Lists vs Arrays

Suppose you have thousands of numerical values to process.



Think: Why might a specialized numerical array be more efficient than a regular Python list?

2. Rows and Columns

A table contains rows and columns, while an image contains pixels arranged in multiple dimensions.

Think: How could a data structure represent both types of information?

3. Mathematical Operations

Suppose two arrays contain corresponding values.

Think: What should happen when you add, multiply, or compare the two arrays?

Prerequisite Concepts

You should be familiar with:

- Python variables and data types
- Python lists
- Basic arithmetic operations
- Basic indexing
- Basic understanding of rows and columns

Core Concepts

1. NumPy

NumPy is a Python library designed for numerical computing. Its main data structure is the `ndarray`, which allows efficient storage and manipulation of numerical data.



2. NumPy Array (ndarray)

An ndarray is a multidimensional array used to store numerical values. Arrays can have one, two, three, or more dimensions depending on the structure of the data.

3. Array Dimensions

The **dimension (ndim)** represents the number of axes in an array. A one-dimensional array resembles a list, while a two-dimensional array resembles a table.

4. Shape

The **shape** of an array describes the number of elements along each dimension. For example, an array with shape (2, 3) contains 2 rows and 3 columns.

5. Size and Data Type

The **size** represents the total number of elements in an array, while dtype indicates the type of data stored, such as integers or floating-point numbers.

6. Creating Arrays

NumPy provides functions such as `np.array()`, `np.ones()`, `np.zeros()`, and `np.arange()` for creating arrays. Random arrays can also be generated using functions from `np.random`.

7. Indexing and Slicing

NumPy allows individual elements or portions of arrays to be accessed using indexing and slicing. Multidimensional arrays can be accessed using indices for each dimension.



8. Arithmetic Operations

NumPy supports element-wise operations such as addition, subtraction, multiplication, division, and exponentiation. These operations can be performed directly on compatible arrays.

9. Aggregation

Aggregation performs an operation on multiple values to produce a summary value. Common operations include sum, mean, max, min, std, and var.

10. Standard Deviation and Variance

Standard deviation measures how spread out values are from their mean, while **variance** measures the average squared deviation from the mean. Higher values indicate greater spread in the data.

11. Reshaping

Reshaping changes the structure of an array without changing its data. The total number of elements must remain the same after reshaping.

12. Transpose

Transposing an array switches its axes, such as converting rows into columns in a two-dimensional array. In NumPy, the transpose of an array can be obtained using `.T`.

13. Element-wise Multiplication

Element-wise multiplication multiplies corresponding elements of two compatible arrays. In NumPy, this operation is performed using the `*` operator.

14. Dot Product

The dot product performs matrix or vector multiplication according to mathematical rules. The dimensions of the participating arrays must be compatible for the operation.

15. Comparison Operators

NumPy supports comparisons such as $>$, $<$, $>=$, $<=$, and $==$. These operations produce Boolean arrays indicating whether each comparison is true or false.

16. Sorting Arrays

NumPy provides functions such as `np.sort()` and `np.argsort()` for arranging array elements and obtaining their sorted indices. Functions such as `np.argmin()` and `np.argmax()` identify positions of minimum and maximum values.

17. NumPy and Pandas

NumPy arrays can be used to create Pandas DataFrames for tabular data analysis. This demonstrates how NumPy numerical structures can work together with other Python data-analysis tools.

18. NumPy and Images

Images can be loaded as NumPy arrays, where pixel information is represented numerically. The **shape**, **size**, and **dimensions** of the resulting array describe the structure of the image.

Guided Reading Questions

1. What is an ndarray, and why is it important in NumPy?
2. What is the difference between shape, size, and ndim?
3. How can NumPy arrays be created using np.array(), np.ones(), and np.zeros()?
4. What is the difference between element-wise multiplication and the dot product?
5. Why is reshaping useful when working with multidimensional arrays?
6. What information is provided by standard deviation and variance?
7. How can NumPy be used to represent an image?

Reflection Activity

Write 100–150 words explaining why NumPy arrays are useful for machine learning. Consider how dimensions, mathematical operations, aggregation, reshaping, and numerical efficiency help in handling ML datasets.

Self-Assessment

1. The main data structure of NumPy is called _____.
2. The attribute used to determine the number of dimensions of an array is _____.
3. The attribute that gives the total number of elements is _____.
4. The function used to calculate the mean of an array is _____.
5. The function used to create an array of zeros is _____.
6. The operation $a * b$ performs _____ multiplication between compatible NumPy arrays.
7. The operation used for matrix multiplication in the notebook is _____.
8. The .T attribute is used to _____ an array.
9. np.argmax() returns the position of the _____ value.
10. Images can be represented in Python as _____ arrays.



Key Takeaway

NumPy provides the fundamental numerical array structure and operations required for machine learning. Understanding **arrays, dimensions, indexing, mathematical operations, aggregation, reshaping, dot products, and sorting** provides an essential foundation for working with ML datasets and algorithms.

